A messaging system requires two main things: a message broker and a protocol. The Advanced Message Queuing Protocol (AMQP) ensures interoperability across platforms and reliable message delivery. It has become widely used in modern architectures, and it's a good fit in the cloud, where we need resilience, loose coupling, and scalability. RabbitMQ is a popular open source message broker that relies on AMQP and provides flexible asynchronous messaging, distributed deployment, and monitoring. Recent RabbitMQ versions have also introduced event streaming features.

Spring provides broad support for the most-used messaging solutions. The Spring Framework itself has built-in support for the Java Message Service (JMS) API. The Spring AMQP project (https://spring.io/projects/spring-amqp) adds support for this messaging protocol and provides integration with RabbitMQ. Apache Kafka is another technology that has become increasingly used in the last few years, such as to implement the event sourcing pattern or real-time stream processing. The Spring for Apache Kafka project (https://spring.io/projects/spring-kafka) provides that integration.

This section will cover the fundamental aspects of the AMQP protocol and RabbitMQ, which is the message broker we'll use to implement messaging in the Polar Bookshop system. On the application side, we'll use Spring Cloud Stream, which offers convenient and robust integration with RabbitMQ by relying on the Spring AMQP project.

10.2.1 Understanding AMQP for messaging systems

When using an AMQP-based solution like RabbitMQ, the actors involved in the interaction can be categorized as follows:

- *Producer*—The entity sending messages (publisher)